IMPLEMENTASI SINGLE LINKED LIST PADA DATA TRAVEL

Laporan Tugas Besar Struktur Data

1st Priska Silvia Ferantiana
NIM 122450053

2nd Berliana Enda Putri
NIM 122450065

3rd M Deriansyah Okutra
NIM 122450101

4th Fayyaza Aqila Syafitri Achjar
NIM 122450131

5th Daffa Ahmad Naufal
NIM 122450137

Abstrak—Single Linked List adalah turunan dari struktur data Linked List yang merupakan sekumpulan data linier yang saling terhubung satu sama lain. Dari sekian banyak kegunaan Single Linked List, salah satunya adalah untuk memudahkan dalam melakukan pengolahan data sehingga didapatkan analisis data yang sesuai dan mudah dipahami. Misalnya dalam pengoperasian data perjalan, dsb. Library yang digunakan adalah Pandas dan Numpy dengan menerapkan konsep Object Oriented Programming (OOP) atau Program Basis Objek (PBO). Penggunaan single Linked List dalam pengolahan data ini diharapkan dapat membantu dan mempermudah analisis data.

Kata kunci—analisis data, Single Linked List, struktur data, travel

I. PENDAHULUAN

A. LATAR BELAKANG

Linked List adalah salah satu bentuk dari struktur data, yang berisikan sekumpulan data yang lebih dikenal dengan *node* dan tersusun secara berurutan, saling sambung-menyambung dan saling terhubung satu sama lain [1]. Setiap *node* pada *Linked List* mempunyai *field* yang berisi *pointer* ke *node* berikutnya dan juga berisi data. *Node* terakhir pada *Single Linked List* akan menunjuk ke NULL atau *node* kosong yang akan digunakan untuk menciptakan kondisi berhenti. Pada penerapannya, *Linked List* kemudian dibagi menjadi beberapa macam, yaitu *Single Linked List* dan *Double Linked List*.

Single Linked List, sesuai dengan namanya adalah suatu kumpulan *node* yang saling berhubungan dengan *node-node* lainnya melalui sebuah panah penunjuk yang disebut *pointer* dan hanya memiliki satu arah saja tanpa bisa melakukan operasi bolak-balik seperti pada *Double Linked List*[2]. *Single Linked List*, sebagaimana dengan *Linked List*, memiliki komponen berupa *head*, *tail*, dan *size*(0). *Head* berfungsi untuk mencatat anggota pertama dari list yang di deskripsikan, *tail* mencatat anggota sebelumnnya dari *node* yang dipanggil atau *node* yang ditunjuk saat ini, sedangkan *size*(0) berfungsi untuk mencatat panjang data atau banyaknya data dari suatu *list* yang dideskripsikan atau dipanggil.

Dalam laporan ini, dataset yang digunakan berjudul ”*Travel Details Dataset”*. *Travel* sendiri diartikan sebagai suatu perjalanan dengan jarak dan kurun waktu tertentu menuju sebuah tempat asing yang belum pernah dikunjungi [3]. Data yang tercantum dalam dataset tersebut meliputi destinasi perjalanan, tanggal dimulai dan berakhir nya perjalanan, durasi perjalanan, tipe akomodasi beserta biayanya, tipe transportasi dan biayanya, serta nama, usia, gender, dan kebangsaan dari orang yang melakukan perjalanan.

Menggunakan *Single Linked List*, data yang diproses di dalam nya diharapkan dapat dianalisis dengan lebih mudah menggunakan fungsi-fungsi atau metode-metode yang ada di dalam *Single Linked List*.

B. RUMUSAN MASALAH

Adapun rumusan masalah yang terkandung dalam latar belakang diatas, yaitu :

- Apakah pengolahan dan analisis data menjadi lebih mudah menggunakan *Single Linked List*?
- Apakah pengolahan data menjadi lebih efektif setelah penerapan *Single Linked List*?

C. DESKRIPSI DATA

Data yang digunakan dalam pengujian *Single Linked List* ini adalah data detail perjalanan yang didapat dari situs dataset, *Kaggle*. Ada kurang-lebih 12 variabel yang terdapat di dalam dataset ini dan tentu saja, tidak semua variabelnya akan digunakan. Dataset yang dipilih berjudul “*Travel Details Dataset*” dimana variabel yang tercantum di dalamnya berisi variabel destinasi perjalanan, tanggal dimulai dan berakhir nya perjalanan, durasi perjalanan, tipe akomodasi beserta biayanya, tipe transportasi dan biayanya, serta nama, usia, gender, dan kebangsaan dari orang yang melakukan perjalanan.

Untuk memasukkan dataset ke dalam program, file dataset terlebih dahulu diubah menjadi file CSV (*Comma Separated Value*). Dataset ini terdiri dari 138 baris data dan 12 value atau variabel. Kemudian, dilakukan konversi variabel sehingga variabel yang tersisa hanya 7 variabel dari

12 variabel, yaitu umur, gender, kebangsaan, tipe akomodasi, biaya akomodasi, tipe transportasi, dan biaya transportasi.

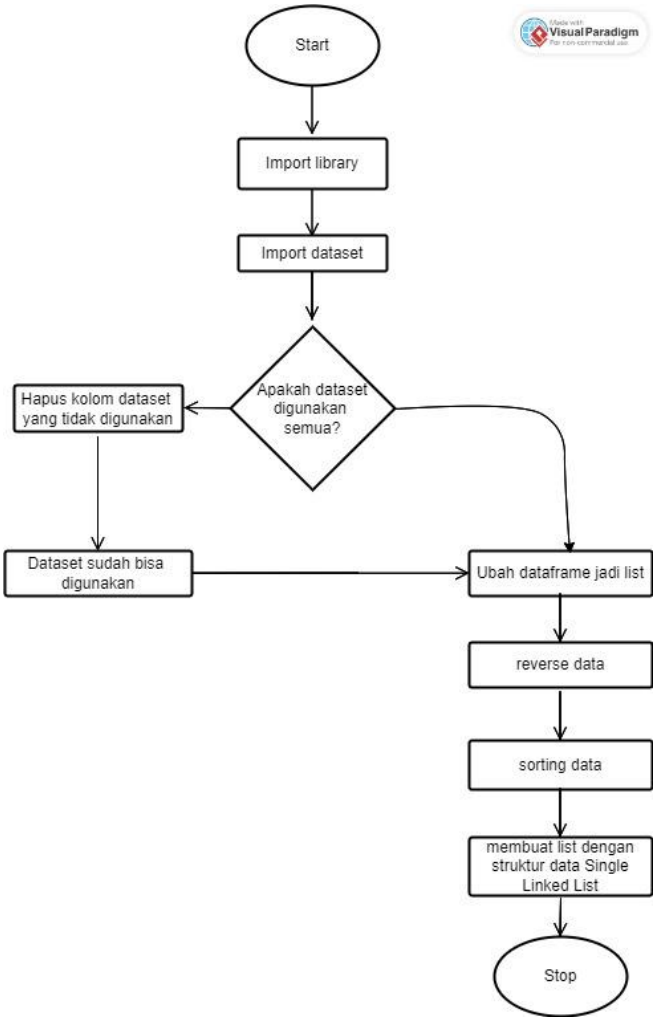
.

II. RANCANGAN SISTEM

A. GAMBARAN UMUM

Pengolahan data dilakukan dengan menggunakan *library Pandas* untuk membaca file CSV dari dataset yang telah diunduh serta menerapkan struktur data *Single Linked List*sebagai metode utama dalam kasus ini, dan juga menggunakan atribut-atribut lain seperti menggunakan *Object Oriented Programming* (OOP), *Numpy*, dan *Pandas*. Dataset yang digunakan diimplementasikan menggunakan struktur data *Single Linked List.*

B. FLOWCHART SYSTEM



Gambar 1

Flowchart (diagram alir) adalah sebuah diagram berupa simbol-simbol dan garis yang merepresentasikan proses dalam program. *Flowchart* menampilkan langkah atau urutan proses yang disimbolkan dalam berbagai bentuk dan secara sistematis menghubungkan langkah-langkah tersebut menggunakan tanda panah.[4].

Gambar di atas merupakan *flowchart* dari program *Single Linked List*. Program ini menggunakan dataset *Travel Details. Flowchart* dimulai dengan symbol oval sebagai awal dari permulaan program. Kemudian, ada dua kotak proses yang masing-masing menyatakan *input library* dan *input dataset*. Setelah itu, *flowchart* memiliki suatu keputusan (digambarkan dalam bentuk *diamond*) yang memeriksa *input* dataset yang diberikan akan digunakan semua atau tidak. Jika iya, program melanjutkan ke proses mengubah *dataframe* menjadi *list*. Lalu melakukan *reverse* data dan dilanjutkan dengan *sorting* data. Namun, jika *input* tidak sesuai, program akan mengarah ke langkah yang meminta pengguna untuk menghapus kolom dataset yang tidak digunakan. Lalu setelah menghapus dataset yang tidak digunakan, dataset yang digunakan akan dilanjutkan dengan mengubah *dataframe* menjadi *list* dan seterusnya. Hingga *flowchart* diakhiri dengan simbol oval sebagai penanda bahwa program telah selesai.

C. DEPENDENCY / LIBRARY

Library yang kami gunakan adalah *Numpy* dan *Pandas* dengan tambahan konsep pemrogramana OOP (*Object Oriented Programming*).

Numpy (Numerical *Python*) digunakan dalam bahasa pemrograman *Python*. *Numpy* biasanya dipakai dalam komputasi ilmiah, terutama dalam analisis data [4]. Tujuan utama numpy untuk menjadi wadah untuk mentransmisikan data antar algoritma, *Numpy* lebih unggul dan efisien penyimpan data dan memanipulasi data dibandingkan yang lainnya. Fitur-fitur OOP antara lain yaitu, *Encapsulation, Inheritance*, dan *Polymorphism*. *Encapsulation*, artinya penggabungan antara atribut dan metode yang mengolahnya. Sedangkan *Inheritance*, adalah penurunan sifat (atribut dan metode) dari kelas *parent* ke kelas *child*. Hal ini berarti OOP mendukung konsep *code reuse* yaitu data yang ada pada kelas parent dapat dikenali oleh kelas *child*. Kemudian, *Polymorphism* artinya sebuah objek memiliki kemampuan untuk bekerja dengan berbagai bentuk..

Pandas adalah perpustakaan (*library*) dalam bahasa pemrograman *Python* yang dirancang untuk melakukan analisis data yang memiliki izin dan persetujuan dari BSD dan bersifat *open source* yang menyediakan struktur data dan analisis data yang mudah digunakan [5]. *Pandas* menyediakan struktur data yang dirancang untuk membantu pekerjaan yang berkaitan dengan struktur data agar menjadi lebih mudah. *Pandas* dikembangkan menggunakan konsep yang dikenalkan oleh *Numpy*, oleh karena itu pengetahuan akan *Numpy* adalah syarat dasar untuk menganalisis data.

Object Oriented Programming (OOP) adalah pemrograman berbasis objek (PBO). OOP mampu menerapkan konsep dari objek yang terdiri dari atribut (variable) dan metode (prosedur).

D. DAFTAR FUNGSIONAL

Adapun fungsi-fungsi yang kami gunakan untuk melakukan pemrograman ini adalah :

- def _init_()
  Digunakan untuk menginisialisasi objek dari *class* yang memiliki parameter di dalamnya.
- def sort()
  Digunakan untuk mengurutkan elemen-elemen yang ada dalam struktur data.
- def measure()
  Dapat digunakan untuk berbagai keperluan, tergantung implementasinya pada suatu program.
- def ada()
  Digunakan dalam suatu *class Linked List* untuk mencari apakah nilai data ada di dalam *Linked List* atau tidak.
- def add_at_beginning()
  Digunakan untuk melakukan penambahan elemen baru di awal struktur data.
- def add_at_end()
  Digunakan untuk menambah elemen baru di akhir struktur data.
- def display()
  Digunakan untuk mencetak atau menampilkan isi dari suatu struktur data.
- def delete_at_end()
  Digunakan untuk menghapus elemen terakhir dari struktur data.
- def delete_at_beginning()
  Digunakan untuk menghapus elemen pertama dari suatu struktur data.
- def delete_at_middle()
  Digunakan untuk menghapus elemen tengah dari suatu struktur data.
- def reverse()
  Digunakan untuk membalik urutan elemen dari suatu struktur data.
- def append_to_list()
  Digunakan untuk menambahkan elemen baru ke dalam suatu list atau array.
- def print_list()
  Digunakan untuk mencetak atau menampilkan isi dari suatu list atau array.

III. HASIL PROGRAM

```
# Mengimpor pustaka NumPy sebagai np dan pustaka Pandas sebagai pd
import numpy as np
import pandas as pd

# Membaca file CSV yang berisi data perjalanan dari path tertentu dan menyimpannya ke dalam variabel df
# '/content/Travel details dataset.csv' adalah path (lokasi) dari file CSV yang akan dibaca
df = pd.read_csv("/content/Travel details dataset.csv")
df
```

Gambar 2

Pada Gambar 1 *Import numpy as np* berfungsi untuk memasukan pustaka atau *library Numpy* ke dalam program python dan memberi alias '*np*' sebagai sebuah *library* di dalam *Python* yang berlisensi BSD dan menyediakan struktur data yang mudah digunakan.

Import pandas as pd berfungsi untuk memasukan pustaka *Pandas* ke dalam program *Python* dan memberi alias '*pd*' untuk memudahkan pemanggilan dan melakukan perhitungan saintifik seperti matriks, aljabar, dan lainnya.

df = pd.read_csv berfungsi untuk membaca file csv-nya dan menampilkan isi dari dataset tersebut.

```
df.head # Menampilkan beberapa baris pertama (secara default lima baris pertama) dari DataFrame df
```

Gambar 3

df.head berfungsi untuk menampilkan beberapa baris pertama dari *dataframe*. Jika ingin menampilkan jumlah baris tertentu, dapat menyertakan argumen beberapa baris. *Head* dapat dengan cepat melihat struktur dan isi awal dari *dataframe* tanpa menampilkan seluruh *dataframe*

```
df=df.drop(columns=["Destination","Start date","End date","Trip ID", "Traveler name","Duration (days)"], axis=0)
df
```

Gambar 4

df = df.drop berfungsi untuk menghapus baris atau kolom dari *dataframe* agar sesuai dengan kebutuhan yang diperlukan.

```
class Node:
    def __init__(self, Traveler_age, Traveler_gender, Travel_nasionality, Accomodation_type, Accomodation_cost, Transportation_type, Transportation_cost):
        self.Traveler_age = Traveler_age  # Usia traveler
        self.Traveler_gender = Traveler_gender  # Jenis kelamin traveler
        self.Travel_nasionality = Travel_nasionality  # Kebangsaan traveler
        self.Accomodation_type = Accomodation_type  # Tipe akomodasi yang digunakan oleh traveler
        self.Accomodation_cost = Accomodation_cost  # Biaya akomodasi traveler
        self.Transportation_type = Transportation_type  # Tipe transportasi yang digunakan oleh traveler
        self.Transportation_cost = Transportation_cost  # Biaya transportasi traveler
        self.next = None  # Pointer ke node selanjutnya dalam linked list
```

Gambar 5

Class node merupakan *Object Oriented Programming* (OOP) dengan metode *def __init__()* yang menjabarkan umur, jenis kelamin, dan kebangsaan dari *traveler*, serta tipe akomodasi, biaya akomodasi, tipe transportasi, dan biaya transportasi yang digunakan oleh *traveler*. Pada setiap ujung-ujung *node*, terdapat *pointer* yang mengarah ke *node* selanjutnya dalam *Linked List*

```
class LinkedList: #kelas linked list
    def __init__(self):
        self.head = None #head belum ada

    def sort(self):
      if self.head is None:  # Jika linked list kosong, tidak perlu dilakukan sorting
        return

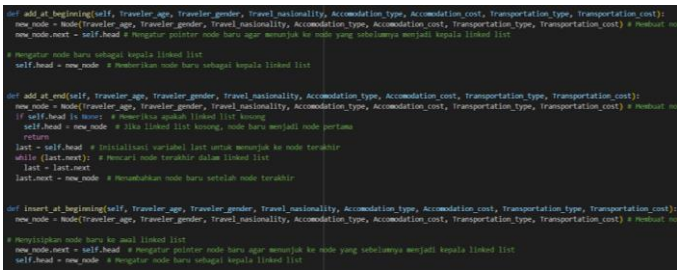
      last = self.head  # Inisialisasi variabel last untuk menunjuk ke node terakhir
      while (last.next):  # Mencari node terakhir dalam linked list
        last = last.next

      current = self.head  # Inisialisasi variabel current untuk iterasi pertama pada linked list
      while current.next:  # Iterasi pada linked list
        if current.data > current.next.data:  # Membandingkan data node saat ini dengan data node berikutnya
            # Jika data node saat ini lebih besar dari data node berikutnya, tukar posisinya
            current.data, current.next.data = current.next.data, current.data
        current = current.next  # Pindah ke node berikutnya
```

Gambar 6

*Class Linked List*merupakan *Object Oriented Programming* (OOP) dengan metode *def __init__()* dan *def*

sort(). Terdapat perulangan menggunakan *if self.head* yang berfungsi apabila *Linked List*nya kosong maka tidak perlu dilakukan sorting. Adapun *last.next* digunakan untuk inisialisasi variabel last yang menunjuk ke node terakhir, yang kemudian diikuti oleh perulangan *while*.

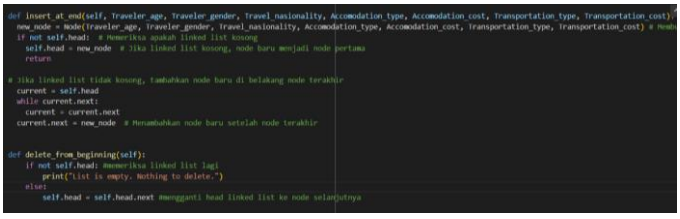


Gambar 7

Metode *def add_at_beginning()* digunakan untuk menambahkan data di awal suatu struktur data, pada *def add_at_beginnning*() terdapat *self.head* yang berguna untuk mengatur pointer *node* baru agar merujuk ke *node* yang sebelumnya menjadi kepala *Linked List*.

Metode *def add_ar_end()* digunakan untuk menambahkan elemen kedalam *list*, didalamnya terdapat *if self.head is none* yang berfungsi untuk memeriksa apakah *Linked List*kosong atau tidak dan akan membentuk *node* baru.

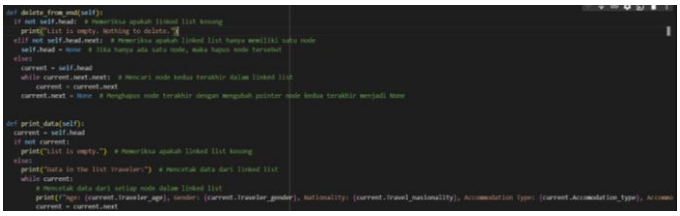
Metode *def insert_at_beginning()* digunakan untuk menerima beberapa parameter dan menambahkan *node* baru. Didalamnya terdapat *new_node.next = self.head* yang digunakan untuk mengatur *pointer node* agar menunjuk ke *node* yang sebelumnya menjadi kepala *Linked List*.



Gambar 8

Metode *def insert_at_end()* digunakan untuk menyisipkan *node* baru. Didalamnya terdapat *if not self .head* yang digunakan untuk memeriksa apakah *Linked List*kosong atau tidak. Jika *Linked List*tidak kosong, maka akan ditambahkan *node* baru di belakang *node* terakhir.

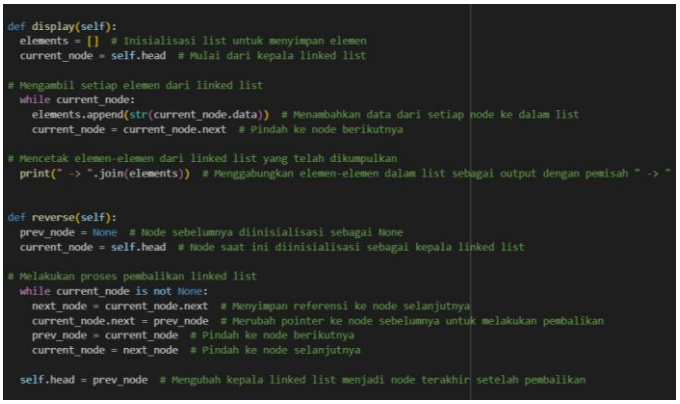
Metode *def delete_from_beginning()* digunakan untuk menghapus elemen atau *node* pertama. Didalamnya terdapat *if not self.head* yang digunakan untuk memeriksa *Linked List*. Kemudian, *else self.head = self.head.next* yang digunakan untuk mengganti *head Linked List*ke *node* selanjutnya.



Gambar 9

Metode *def delate_from_end()* digunakan untuk menghapus elemen atau *node* pada bagian terakhir dari list. Didalamnya terdapat percabangan *if, elif*, dan *else* untuk memeriksa *Linked List*nya hanya memiliki satu *node* atau tidak. Jika *Linked List*hanya memiliki satu *node*, maka *node* tersebut akan dihapus

Metode *def print_data()* digunakan untuk menampilkan data yang ada pada *Linked List*tersebut.



```
def display(self):
    elements = []  # Inisialisasi list untuk menyimpan elemen
    current_node = self.head  # Mulai dari kepala linked list

# Mengambil setiap elemen dari linked list
    while current_node:
        elements.append(str(current_node.data))  # Menambahkan data dari setiap node ke dalam list
        current_node = current_node.next  # Pindah ke node berikutnya

# Mencetak elemen-elemen dari linked list yang telah dikumpulkan
    print(" -> ".join(elements))  # Menggabungkan elemen-elemen dalam list sebagai output dengan pemisah " -> "


def reverse(self):
    prev_node = None  # Node sebelumnya diinisialisasi sebagai None
    current_node = self.head  # Node saat ini diinisialisasi sebagai kepala linked list

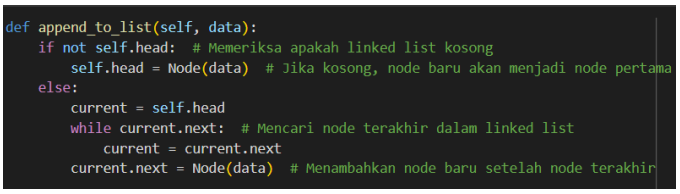
# Melakukan proses pembalikan linked list
    while current_node is not None:
        next_node = current_node.next  # Menyimpan referensi ke node selanjutnya
        current_node.next = prev_node  # Merubah pointer ke node sebelumnya untuk melakukan pembalikan
        prev_node = current_node  # Pindah ke node berikutnya
        current_node = next_node  # Pindah ke node selanjutnya

    self.head = prev_node  # Mengubah kepala linked list menjadi node terakhir setelah pembalikan
```

Gambar 10

Metode *def display()* digunakan untuk menambahkan elemen dan menyimpan elemen yang menggunakan *list append* untuk menambahkan data dari setiap *node*.

Metode *def reverse()* digunakan untuk mengurutkan elemen, didalamnya terdapat *prev_node* dan *current_node* yang diinisiasikan sebagai kepala Linked List. Terdapat proses pembalikan *Linked List* dengan menggunakan *while current_node*.



```
def append_to_list(self, data):
    if not self.head:  # Memeriksa apakah linked list kosong
        self.head = Node(data)  # Jika kosong, node baru akan menjadi node pertama
    else:
        current = self.head
        while current.next:  # Mencari node terakhir dalam linked list
            current = current.next
        current.next = Node(data)  # Menambahkan node baru setelah node terakhir
```

Gambar 11

Metode *def append_to_list()* digunakan untuk menambahkan elemen baru ke dalam *Linked List* yang di dalamnya terdapat perulangan *if* dan *else* untuk memeriksa apakah *Linked List* nya kosong dan mencari *node* terakhir dalam *Linked List*.

```
df =df.values.tolist() # Mengonversi DataFrame menjadi list menggunakan metode values.tolist()
df
```

Gambar 12

df = df.values.tolist digunakan untuk mengonversi *dataframe* menjadi *list* menggunakan metode *def values.tolist().*

```
df.reverse() # Membalik urutan elemen dalam list menggunakan metode reverse()
df # Menampilkan list setelah pengurutan dibalik
```

Gambar 13

df.reverse digunakan untuk membalik urutan elemen dalam *list* menggunakan metode *reverse* dan menampilkan *list* setelah pengurutan dibalik.

```
df.sort() # Melakukan pengurutan elemen dalam list atau struktur data dengan metode sort()
df # Menampilkan list atau struktur data setelah diurutkan
```

Gambar 14

df.sort digunakan untuk melakukan pengurutan elemen dalam *list* atau struktur data dengan metode *sort* dan menampilkan *list* setelah diurutkan.

```
#menjalankan program
linked_list = LinkedList()

while True:
    print("\nMenu:")#kode menampilkan menu
    print("1. Insert Data at the Beginning")
    print("2. Insert Data at the End")
    print("3. Delete Data from the Beginning")
    print("4. Delete Data from the End")
    print("5. Print All Data")
    print("6. Quit")

    choice = input("Enter your choice (1/ 2/ 3/ 4/ 5/ 6): ")#memanggil kode menu dengan angka
```

Gambar 15

Pada program diatas kode akan dijalankan dan ditampilkan dengan menggunakan *print* dan memanggil kode dengan menggunakan *input.*

```
if choice == "1": #menambahkan data di awal
    traveler_age = int(input("Masukkan umur: "))
    Traveler_gender = input("Masukkan jenis kelamin: ")
    Travel_nasionality = input("Masukkan kebangsaan: ")
    Accomodation_type = input("Jenis penginapan: ")
    Accomodation_cost = int(input("Masukkan pengeluaran penginapan: $"))
    Transportation_type = input("Masukkan jenis transportasi: ")
    Transportation_cost = int(input("Masukkan pengeluaran transportasi: $"))
    linked_list.add_at_beginning(Traveler_age, Traveler_gender, Travel_nasionality, Accomodation_type, Accomodation_cost, Transportation_type, Transportation_cost)
elif choice == "2": #menambahkan data di akhir
    Traveler_age = int(input("Masukkan umur: "))
    Traveler_gender = input("Masukkan jenis kelamin: ")
    Travel_nasionality = input("Masukkan kebangsaan: ")
    Accomodation_type = input("Jenis penginapan: ")
    Accomodation_cost = int(input("Masukkan pengeluaran penginapan: $"))
    Transportation_type = input("Masukkan jenis transportasi: ")
    Transportation_cost = int(input("Masukkan pengeluaran transportasi: $"))
    linked_list.add_at_end(Traveler_age, Traveler_gender, Travel_nasionality, Accomodation_type, Accomodation_cost, Transportation_type, Transportation_cost)
elif choice == "3": #menghapus data dari awal
    linked_list.delete_from_beginning()
elif choice == "4": #menghapus data dari akhir
    linked_list.delete_from_end()
elif choice == "5": #mencetak semua data yang telah ditambahkan
    linked_list.print_data()
elif choice == "6": #memberhentikan program
    break  #keluar looping
else:
    print("Invalid choice. Please try again.") #kode tidak akan dijalankan
```

If choice digunakan untuk menambahkan data di awal dan memasukan *input* umur, jenis kelamin, kebangsaan, jenis akomodasi, biaya akomodasi, jenis transportasi, dan biaya transportasi.

IV. KESIMPULAN

Single Linked List merupakan sebuah variabel *pointer* untuk menyimpan banyak data dengan metode *Linked List*, *Single Linked List* adalah sekumpulan *node* yang saling terhubung dengan *node* lainnya menggunakan *pointer*. *Single Linked List* hanya memiliki satu arah dan tidak bisa melakukan alur bolak balik. Penerapan pada S*ingle Linked List* untuk data *travel* bertujuan untuk mengetahui data-data orang yang akan bepergian dan mengurutkan berdasarkan umur, jenis kelamin, kebangsaan, penginapan, jenis transportasi. Dalam menggunakan S*ingle Linked List* dapat menggunakan destinasi baru, sehingga terbukti bahwa analisis data menjadi lebih mudah dilakukan dan lebih efisien serta memudahkan akses untuk melihat dari data-data S*ingle Linked List*.

V. DAFTAR PUSTAKA

[1] A. A. N. Siregar, “Pengertian Linked Object,” Fakultas Komputer Universitas Mitra Indonesia, Bandar Lampung.

[2] A. K. Putra, “Single Linked List,” Fakultas Komputer Universitas Mitra Indonesia, Bandar Lampung.

[3] Oxford University Press, “Oxford English Dictionary,” Oxford University, [Online]. Available: https://www.oed.com/search/advanced/Entries?textTermText0=travel&textTermOpt0=WordPhrase&dateOfUseFirstUse=false&page=1&sortOption=DateNewFirst. [Diakses 10 Desember 2023].

[4] A. Karim dan E. Purba, “Sistem Informasi Pendataan Penduduk Kelurahan Kampung Mesjid Berbasis Web,” dalam *Seminar Nasional Sains & Teknologi Informasi (SENSASI)*, Rantauprapat, 2018.

[5] S. Atharva dan V. Shubham, “Scientific Computing and Data Analysis using NumPy and Pandas,” *International Research Journal of Engineering and Technology (IRJET)*, pp. 1334-1346, 2020.

[6] D. Y. Chen, Pandas for Everyone: Python Data Analysis, Addison-Wesley Professional, 2017.